

Investigation Note:

I explored the dataset of the support tickets which had information about certain fields task 0 was to figure out which of these are reliable (either IDs or something provided as an input from the end user) and which of these can be inferred fields. After observation it came to my notice that the fields for category, subcategory and the priority are the ones that were **missing at lot of places** and can be easily inferred by reading the description etc. in the ticket.

These were the fields that can be generated from the LLM if a suitable persona was provided. I generated a prompt for missing data filling and used the surrounding information as a context to the LLM to generate the missing values, structured output method was used to generate the outcomes in reliable ways and added to the solution.

Next up was the **recurrence pattern**, the main cue I found here was grouping the tickets by customer ID – once done it was easy to pass the list of tickets by a single customer to the LLM and check if any of the tickets are related to any other. **Note:** Here I have used a format {Ticket 001 is follow-up of Ticket 002} kind of string but later this can be converted to forms like graphs etc. for easier readability and linking. **Actually, this task is one of the tasks in the BERT model paper that says one of the tasks of BERT are to identify semantic relation between two statements.** Which made the use case even stronger for LLM.

Next was summarization, for this I passed each ticket to the LLM and an outcome as a summary was generated. I completely agree that this is a time and token consuming mechanism but this can be added as a filter when the data enters the system and done only once for each ticket, **moreover smaller local models like QWEN 0.5B can do it for free** so I believe it's not much of a problem to use per ticket 1 call to the LLM here. Also, **if provided with a set of pervious solutions this can be further elevated to a feature that will generate a base plan to tackle the issue.**

Proof of Concept:

The implementation of the POC was completed in 4 hours hence some of the parts just demonstrate the ideation and many optimizations can be done at implementation time.

1. The data from the csv is converted and loaded to a python dictionary, at the same time it is formatted as {Customer_ID : [List of tickets associated.]}
2. Each ticket is passed through and checked if the inferred values are present or missing, if the values the values are missing then an LLM call is made to fill the missing the value.
3. The completed data set is then passed through for recurrence checking, here each of the customer's list of tickets is passed through and checked for semantic follow-ups and connections, if they are connected then they are marked so.
4. Finally, the tickets are passed for summarization which is mostly based on the description of the ticket
5. All the responses are stored in JSON as of now but can be stored in SQL or excel as needed.

NOTE: currently the pipeline is sequential but since there is no dependencies in the three tasks they can be run in parallel in production.

For most of the tasks based on text interpretation free models like QWEN 2.5 0.5 B can be set up and used at no cost and no high RAM. For larger tasks we may need a better model.

Usage of AI:

- Mostly I relied on **Github Copilot**, which is inbuilt for VSCODE, it allows inline completions based on my own previous work.
- I have not used AI for ideation and design work; I usually like to generate a pseudo code and then let copilot assist or just resolve the issues or optimize the pipeline using AI.
- The Readme file was created with AI as using AI for documentation and repeating code writing is something I prefer.